

DSA-2 Quick Notes

Data Structures & Algorithms II · BCSE0401 / BCS301

All 5 Units · NIET Greater Noida · End Semester Exam

How to use these notes:

These are comprehensive but concise short notes covering every topic from all 5 units. Each section has: what the topic IS (theory), how it WORKS (algorithm steps), and key FACTS to write in answers. For 10-mark answers always add a diagram. Bold terms are the ones examiners look for.

UNIT 1 — Trees

1.1 Tree Terminology

A **tree** is a hierarchical, non-linear data structure consisting of nodes connected by edges. Unlike graphs, trees have no cycles and there is exactly one path between any two nodes. A tree with N nodes always has exactly $N-1$ edges.

Structural Terms

Root: topmost node; has no parent

Leaf (External node): node with no children; degree = 0

Internal node: any non-leaf node; has at least one child

Degree of node: number of children a node has

Degree of tree: maximum degree among all nodes

Parent / Child: nodes directly connected; upper is parent

Siblings: nodes sharing the same parent

Ancestor: any node on the path from root to a node

Descendant: any node in the subtree rooted at a node

Measurement Terms

Height of node: edges on longest path from node to a leaf

Height of tree: height of root node

Depth of node: edges from root to that node (root depth = 0)

Level: depth + 1; root is at level 1

Path: sequence of nodes connected by edges

Subtree: a node and all its descendants

Forest: collection of disjoint trees

Edge: connection between parent and child

1.2 Types of Binary Trees

A **Binary Tree** is a tree where every node has at most 2 children (left and right).

Full Binary Tree: every node has EXACTLY 0 or 2 children (never 1). Key theorem: $L = I + 1$ where L =leaves, I =internal nodes. Also: $N = 2L - 1$ and $N = 2I + 1$

Complete Binary Tree: all levels completely filled EXCEPT the last level, which is filled from LEFT to RIGHT. Used in heaps.

Strictly Binary Tree: every node has exactly 0 or 2 children. Same as full binary tree in many textbooks.

Extended Binary Tree (2-tree): all nodes have 0 or 2 children. External (leaf) nodes represent empty positions.

Perfect Binary Tree: all internal nodes have exactly 2 children AND all leaves are at the same level. Max nodes at height $h = 2^{(h+1)} - 1$.

1.3 Tree Traversals

Tree traversal means visiting every node exactly once in a specific order. The three standard traversals differ in when the root is visited relative to subtrees.

Three Traversal Orders

Inorder (LNR): Left subtree → Root → Right subtree.
For a BST, Inorder gives nodes in SORTED ascending order.

Preorder (NLR): Root → Left subtree → Right subtree.
Used to create a copy of the tree.

Postorder (LRN): Left subtree → Right subtree → Root.
Used to delete a tree or evaluate expression trees.

Constructing a Unique Tree

You CANNOT construct a unique binary tree from only one traversal. You need: Inorder + Preorder OR Inorder + Postorder. Preorder + Postorder together do NOT always give a unique tree.

Construction Method

Given Inorder + Preorder:

- First element of Preorder = Root
- Find Root in Inorder array
- Elements left of Root in Inorder = Left subtree
- Elements right of Root in Inorder = Right subtree
- Recursively repeat for each subtree

Given Inorder + Postorder:

- LAST element of Postorder = Root
- Same splitting process in Inorder
- Recursively repeat

1.4 Binary Search Tree (BST)

A **Binary Search Tree** is a binary tree with the BST property: for every node, ALL nodes in its left subtree have smaller values, and ALL nodes in its right subtree have greater values. This property holds recursively for every node in the tree.

Search: Start at root. If target < node, go left. If target > node, go right. If target = node, found. If reach NULL, not found. Time: $O(\log n)$ average, $O(n)$ worst case (skewed tree).

Insert: Search for the position where the key should go (same as search). When you reach a NULL pointer, insert the new node there. Always inserted as a leaf.

Delete — 3 cases:

- Case 1 (Leaf node): Simply remove the node. Set parent's pointer to NULL.
- Case 2 (One child): Replace the node with its only child. Parent now points to the child directly.
- Case 3 (Two children): Find the Inorder Successor (smallest node in right subtree, i.e., go right once then go left as far as possible). Copy its value into the node being deleted. Then delete the inorder successor (which has at most one child — Case 1 or 2).

Time Complexity: Average $O(\log n)$ for all operations. Worst case $O(n)$ for a completely skewed tree (all nodes in one direction — essentially a linked list).

1.5 AVL Trees

An **AVL Tree** (Adelson-Velsky and Landis) is a self-balancing BST. It maintains a Balance Factor (BF) for every node where $BF = \text{Height}(\text{Left Subtree}) - \text{Height}(\text{Right Subtree})$. A valid AVL tree requires $BF \in \{-1, 0, +1\}$ for every node. After every insert or delete, if any node's BF becomes +2 or -2, a rotation is performed to rebalance.

Four Rotation Cases:

LL Rotation (Left-Left): New node inserted in LEFT subtree of LEFT child of the unbalanced node. Fix: Single RIGHT rotation at the unbalanced node.

RR Rotation (Right-Right): New node inserted in RIGHT subtree of RIGHT child. Fix: Single LEFT rotation at the unbalanced node.

LR Rotation (Left-Right): New node inserted in RIGHT subtree of LEFT child. Fix: First LEFT rotate the left child, then RIGHT rotate the unbalanced node. (Double rotation)

RL Rotation (Right-Left): New node inserted in LEFT subtree of RIGHT child. Fix: First RIGHT rotate the right child, then LEFT rotate the unbalanced node. (Double rotation)

How to identify rotation type: Starting from the unbalanced node ($BF=+2$ or -2), look at the direction of the taller subtree (first letter: L if $BF=+2$, R if $BF=-2$). Then look at the direction of the taller subtree of the child (second letter). Together they give LL, LR, RL, or RR.

Height guarantee: AVL tree with n nodes has height $O(\log n)$. All operations (search, insert, delete) are $O(\log n)$ worst case — unlike plain BST.

1.6 B-Trees

A **B-Tree of order m** is a balanced multi-way search tree designed for efficient disk storage. Unlike BSTs, each node can hold multiple keys and have multiple children. B-Trees are the foundation of most database index structures.

Properties of B-Tree of order m :

- Every node has at most m children
- Every node (except root) has at least $\lceil m/2 \rceil$ children
- Root has at least 2 children (if it is not a leaf)
- All leaves appear at the same level (perfectly balanced)
- A node with k children contains exactly $k-1$ keys
- Keys within a node are sorted in ascending order

Insert Algorithm: Always insert into a leaf node. Search for the correct leaf. If leaf is not full ($< m-1$ keys), insert there. If leaf is FULL ($m-1$ keys), SPLIT: median key moves UP to parent, left and right halves remain as two nodes. If parent also becomes full, split propagates up. If root splits, a new root is created and tree height increases by 1.

Delete Algorithm: If key is in leaf and leaf has enough keys — simply remove. If leaf becomes underflowed, borrow from sibling (rotation) or merge with sibling. If key is in internal node, replace with Inorder Predecessor or Successor, then delete from leaf.

Why B-Trees? Minimizes disk reads. A single disk block can store one B-Tree node with hundreds of keys. Tree height stays very low $\rightarrow$ very few disk accesses for any operation.

1.7 Heaps and Heap Sort

A **Binary Heap** is a complete binary tree stored as an array where elements satisfy the heap property. In a Max Heap, every parent $\geq$ its children. In a Min Heap, every parent $\leq$ its children. Array indexing: parent at index i , left child at $2i+1$, right child at $2i+2$ (0-indexed).

Heap Sort Algorithm:

Phase 1 — Build Max Heap: Starting from the last non-leaf node (index $n/2 - 1$), call Heapify on each node going backwards to index 0. Heapify: Compare node with its children. If a child is larger, swap with the largest child. Continue Heapifying down until the heap property is restored. Time: $O(n)$.

Phase 2 — Sort: The root always holds the maximum element. Swap root with the last element. Reduce heap size by 1 (last element is now in its sorted position). Call Heapify on the root to restore heap property. Repeat until only one element remains. Time: $O(n \log n)$.

Key Properties: Time complexity $O(n \log n)$ for ALL cases (best, average, worst). Space complexity $O(1)$ — sorts in-place. Not stable (relative order of equal elements not preserved). Not cache-friendly compared to Quicksort.

Insert into heap: Add element at end. "Bubble up" — compare with parent, swap if larger (max heap), repeat until heap property holds.

Extract-Max: Remove root (max). Put last element at root. Heapify down. Time: $O(\log n)$.

UNIT 2 — Graphs

2.1 Graph Basics & Terminology

A **Graph $G = (V, E)$** is a non-linear data structure consisting of a set of vertices V (nodes) and a set of edges E (connections between vertices). Graphs model relationships between objects — used in maps, social networks, compilers, networks, etc.

Types of Graphs

Undirected Graph: edges have no direction; $(u,v) = (v,u)$

Directed Graph (Digraph): edges have direction; $(u,v) \neq (v,u)$

Weighted Graph: edges carry a numerical weight/cost

Connected Graph: every vertex is reachable from every other

Disconnected Graph: at least one vertex not reachable from another

Null Graph: vertices present but no edges

Complete Graph: every pair of vertices has an edge; K_n has $n(n-1)/2$ edges

DAG: Directed Acyclic Graph — directed graph with no cycles

Key Terms

Degree (undirected): number of edges incident to vertex

In-degree (directed): number of edges coming INTO vertex

Out-degree (directed): number of edges going OUT of vertex

Path: sequence of vertices where each pair is connected by edge

Cycle: path that starts and ends at same vertex

Simple Path: path with no repeated vertices

Connected Component: maximal connected subgraph

Spanning Tree: subgraph with all vertices, $N-1$ edges, no cycle

2.2 Graph Representations

Adjacency Matrix

A $V \times V$ 2D array where $matrix[i][j] = 1$ if edge exists from vertex i to j , else 0. For weighted graphs, store weight instead of 1.

Pros: $O(1)$ edge lookup. Simple implementation. Easy to check if edge exists.

Cons: Always uses $O(V^2)$ space even for sparse graphs with few edges. Traversing all edges of a vertex takes $O(V)$ time.

Best for: Dense graphs where $E \approx V^2$.

Adjacency List

An array of V linked lists. $adjacencyList[i]$ contains all vertices adjacent to vertex i . For weighted graphs, each list node stores (neighbour, weight).

Pros: Space efficient — $O(V+E)$. Traversing neighbours is $O(\text{degree})$.

Cons: Edge lookup is $O(\text{degree})$. More complex to implement.

Best for: Sparse graphs where $E \ll V^2$. Used in most real applications.

2.3 DFS — Depth First Search

DFS explores a graph by going as deep as possible along each branch before backtracking. It uses a stack (or recursion, which uses the call stack internally). Think of it as exploring a maze by always taking the first unexplored path until you hit a dead end.

Algorithm:

- Initialize: create visited[] array (all false), push starting vertex onto stack, mark it visited
- While stack is not empty: pop vertex u , print/process u
- For each unvisited neighbour v of u : mark v visited, push v onto stack
- If a vertex has no unvisited neighbours, it is popped (backtracking occurs)

Time Complexity: $O(V+E)$ — visits every vertex and edge exactly once

Space Complexity: $O(V)$ for visited array + $O(V)$ for stack in worst case

Applications: Detecting cycles in a graph. Topological sorting of DAGs. Finding connected components. Path finding in mazes. Strongly Connected Components (Tarjan's / Kosaraju's algorithm). Solving puzzles with one solution (Sudoku, N-Queens).

2.4 BFS — Breadth First Search

BFS explores a graph level by level — first visits all vertices at distance 1 from source, then all at distance 2, and so on. Uses a Queue (FIFO). Guarantees shortest path (minimum edges) in unweighted graphs.

Algorithm:

- Initialize: create visited[] array (all false), enqueue starting vertex, mark it visited
- While queue is not empty: dequeue vertex u, print/process u
- For each unvisited neighbour v of u: mark v visited, enqueue v
- Continue until queue is empty

Time Complexity: $O(V+E)$

Space Complexity: $O(V)$ for visited array + $O(V)$ for queue (entire frontier stored)

Applications: Shortest path in unweighted graphs. Finding all nodes within k hops. Web crawling (crawl layer by layer). Level-order traversal of trees. Broadcasting in networks. Peer-to-peer networks (BitTorrent). Garbage collection (Cheney's algorithm).

DFS vs BFS Summary:

DFS: Stack, explores deep first, good for cycle detection and connectivity, less memory for deep graphs. **BFS:** Queue, explores wide first, guarantees shortest path (unweighted), more memory (stores whole frontier).

2.5 Minimum Spanning Trees

A **Minimum Spanning Tree (MST)** of a connected, weighted, undirected graph is a spanning tree (includes all vertices, $N-1$ edges, no cycle) with the minimum possible total edge weight. Two main algorithms: Kruskal's and Prim's. Both are greedy.

Kruskal's Algorithm

Approach: Edge-based. Works well for sparse graphs.

Step 1: Sort ALL edges by weight in ascending order.

Step 2: Initialize each vertex as its own set (Union-Find).

Step 3: Pick the smallest edge. If adding it creates a cycle (both endpoints in same set) — SKIP. Otherwise, add to MST and union the two sets.

Step 4: Repeat until $V-1$ edges are selected.

Time: $O(E \log E)$ dominated by sorting.

Cycle detection: Use Union-Find (Disjoint Set Union) data structure — `find()` operation checks if two vertices are in same component.

Prim's Algorithm

Approach: Vertex-based. Grows one tree from a starting vertex.

Step 1: Start with any vertex. Add to MST set.

Step 2: Look at ALL edges connecting MST vertices to non-MST vertices (fringe edges). Pick the one with minimum weight.

Step 3: Add that edge and its endpoint to MST.

Step 4: Repeat until all vertices are in MST.

Time: $O(V^2)$ with adjacency matrix. $O(E \log V)$ with min-heap.

Key diff from Kruskal: Always maintains ONE connected tree and grows it. Kruskal may have disconnected components until they merge.

2.6 Shortest Path Algorithms

Dijkstra's Algorithm — Single Source Shortest Path

Finds the shortest path from a single source vertex to all other vertices in a weighted graph. Works only for graphs with NON-NEGATIVE edge weights.

- Initialize: $\text{dist}[\text{source}] = 0$, $\text{dist}[\text{all others}] = \text{infinity}$, $\text{visited} = \text{empty set}$
- Pick the unvisited vertex u with minimum distance. Mark visited.
- RELAX: For each unvisited neighbour v of u : if $\text{dist}[u] + \text{weight}(u,v) < \text{dist}[v]$, then update $\text{dist}[v] = \text{dist}[u] + \text{weight}(u,v)$
- Repeat until all vertices are visited

Time: $O(V^2)$ with adjacency matrix. Cannot handle negative weight edges.

Bellman-Ford Algorithm — Handles Negative Edges

Finds shortest paths from source even when some edge weights are negative. Can also detect negative weight cycles (if a negative cycle exists, no shortest path is defined).

- Initialize: $\text{dist}[\text{source}] = 0$, $\text{dist}[\text{all others}] = \text{infinity}$
- Repeat $V-1$ times: for every edge (u,v,w) : if $\text{dist}[u] + w < \text{dist}[v]$, update $\text{dist}[v]$
- After $V-1$ iterations, do ONE MORE pass. If any distance still updates, a negative cycle exists → return FALSE

Time: $O(V \times E)$. Slower but more powerful than Dijkstra.

Floyd-Warshall — All Pairs Shortest Path

Finds shortest paths between EVERY pair of vertices. Uses Dynamic Programming. Runs in $O(V^3)$. Can handle negative weights but not negative cycles.

- Init: $D[i][j] = \text{edge weight if edge exists, } 0 \text{ if } i=j, \text{ infinity if no edge}$
- For each intermediate vertex k (from 0 to $V-1$):
 - For every pair (i, j) : $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$
- After processing all k , $D[i][j]$ holds the shortest path from i to j

Show $D[0]$, $D[1]$, $D[2]$... $D[k]$ matrices in exam answer for full marks.

UNIT 3 — Dynamic Programming

3.1 What is Dynamic Programming?

Dynamic Programming (DP) is an algorithmic technique for solving optimization problems by breaking them into smaller overlapping subproblems, solving each subproblem once, and storing the result to avoid redundant computation. It is fundamentally different from Divide & Conquer — in D&C, subproblems are independent; in DP, subproblems **OVERLAP** (the same subproblem appears in multiple places).

Two Required Properties for DP:

- 1. Optimal Substructure:** An optimal solution to the problem contains optimal solutions to its subproblems. Example: The shortest path from A to C via B = shortest path A→B + shortest path B→C.
- 2. Overlapping Subproblems:** The recursive solution recomputes the same subproblems many times. DP stores each result in a table so it is computed only once.

DP vs Greedy vs Divide & Conquer:

Greedy: Makes the locally optimal choice at each step, never revises. Fast but may miss global optimum. **DP:** Considers all possibilities systematically using a table. Slower but guarantees optimal. **D&C:** Splits into **INDEPENDENT** subproblems (no overlap). Mergesort is D&C, not DP.

Two implementations of DP: **Memoization (Top-Down):** Write recursive solution, add a cache — if subproblem already solved, return cached result. **Tabulation (Bottom-Up):** Fill a table iteratively from smallest subproblems upward. Usually preferred in exams.

3.2 0/1 Knapsack Problem

Problem: Given n items each with a weight $w[i]$ and profit $v[i]$, and a knapsack with maximum capacity W — select items to maximize total profit without exceeding capacity. Each item can be taken (1) or left (0) — cannot take fractions (that would be Fractional Knapsack → Greedy).

DP Table Setup: Create table $c[0..n][0..W]$. $c[i][w]$ = maximum profit using first i items with capacity w .

Base Cases: $c[0][w] = 0$ for all w (no items = no profit). $c[i][0] = 0$ for all i (no capacity = no profit).

Recurrence Relation:

If $w[i] \leq w$ (item fits): $c[i][w] = \max(c[i-1][w], v[i] + c[i-1][w - w[i]])$

If $w[i] > w$ (item does not fit): $c[i][w] = c[i-1][w]$

How to backtrack and find which items are selected: Start at $c[n][W]$. If $c[i][w] \neq c[i-1][w]$, then item i is included — move to $c[i-1][w-w[i]]$. Else item i is not included — move to $c[i-1][w]$. Continue until $i=0$.

Time: $O(nW)$. **Space:** $O(nW)$. Note: Fractional Knapsack (can take fractions) is solved by Greedy — sort by profit/weight ratio.

3.3 Longest Common Subsequence (LCS)

A **subsequence** is a sequence derived from another by deleting some elements without changing the order of remaining elements. Unlike a substring, elements need NOT be contiguous. LCS finds the longest subsequence common to two strings X and Y .

DP Table: $L[i][j]$ = length of LCS of $X[1..i]$ and $Y[1..j]$.

Base Cases: $L[i][0] = 0$ and $L[0][j] = 0$ (empty string has LCS of 0).

Recurrence:

If $X[i] = Y[j]$: $L[i][j] = L[i-1][j-1] + 1$

If $X[i] \neq Y[j]$: $L[i][j] = \max(L[i-1][j], L[i][j-1])$

To find actual LCS string (backtracking): Start at $L[m][n]$. If $X[i]=Y[j]$, this character is in LCS — move diagonally to $L[i-1][j-1]$. Else move in direction of the larger value ($\uparrow$ or $\leftarrow$). Collect characters where you moved diagonally (reverse order for final string).

Time: $O(m \times n)$ where m, n are lengths of X and Y

LCS vs Substring: Substring is contiguous; subsequence is not. "ACE" is a subsequence but not a substring of "ABCDE"

3.4 Matrix Chain Multiplication

Problem: Given a chain of matrices $A_1, A_2, \dots, A_n$, find the optimal way to parenthesize the product to minimize total scalar multiplications. Matrix multiplication is associative — order of matrices is fixed, but parenthesization can vary.

Cost of multiplication: Multiplying a $p \times q$ matrix with a $q \times r$ matrix costs $p \times q \times r$ scalar multiplications.

DP Table: $m[i][j]$ = minimum multiplications to compute $A_i \times A_{i+1} \times \dots \times A_j$.

Base Case: $m[i][i] = 0$ (single matrix, no multiplication needed).

Recurrence (try every possible split point k):

$m[i][j] = \min \text{ over } k \text{ from } i \text{ to } j-1 \text{ of: } m[i][k] + m[k+1][j] + p[i-1] \times p[k] \times p[j]$

where $p[i-1]$ = rows of A_i , $p[k]$ = shared dimension, $p[j]$ = cols of A_j

Fill table diagonally: First compute all chains of length 2, then length 3, then 4, etc.

Time: $O(n^3)$. Space $O(n^2)$.

To reconstruct optimal parenthesization: Maintain a second table $s[i][j]$ = the k that gave minimum cost. Use s to trace back the optimal split.

3.5 Assembly Line Scheduling

Problem: A factory has 2 assembly lines, each with n stations. A car chassis enters one of the lines, passes through n stations, then exits. At each station there is a processing time $a[\text{line}][\text{station}]$. There is a transfer time $t[\text{line}][\text{station}]$ to switch between lines after any station. Find the minimum time to get through the factory.

Recurrence:

$f_1[1] = e_1 + a[1][1]$; $f_2[1] = e_2 + a[2][1]$ (entry times + first station)

$f_1[j] = \min(f_1[j-1] + a[1][j], f_2[j-1] + t[2][j] + a[1][j])$

$f_2[j] = \min(f_2[j-1] + a[2][j], f_1[j-1] + t[1][j] + a[2][j])$

Final = $\min(f_1[n] + x_1, f_2[n] + x_2)$ where x_1, x_2 are exit times

Explanation: $f_1[j]$ = fastest time to reach station j on line 1. You either came from station $j-1$ on line 1 (stay) or switched from line 2 after station $j-1$ (incur transfer cost $t[2][j]$).

Time: $O(n)$ — much better than brute force $O(2^n)$

UNIT 4 — Backtracking & Branch and Bound

4.1 Backtracking — Core Concept

Backtracking is a systematic method for solving problems by incrementally building a solution and abandoning (backtracking) a partial solution as soon as it is determined that the partial solution cannot be extended to a valid complete solution. It is essentially a DFS on the state-space tree.

State-Space Tree: A tree representing all possible states of the solution. Each node represents a partial solution. Root = empty solution. Leaves = complete solutions. Backtracking does DFS on this tree, pruning branches early.

Pruning: When a partial solution violates a constraint, its entire subtree is cut off (pruned). This avoids exploring solutions that cannot succeed — key to backtracking's efficiency.

Bounding Function: A function that determines whether a partial solution can lead to a valid complete solution. Used to decide when to prune.

Three types of backtracking problems:

Decision: Does a solution exist? Answer yes/no.

Optimization: Find the BEST solution among all valid solutions (e.g., minimum cost).

Enumeration: Find ALL valid solutions (e.g., all arrangements of N-Queens).

Backtracking vs Branch & Bound: Backtracking prunes branches that are INFEASIBLE (constraint violated). Branch & Bound also prunes branches that are SUBOPTIMAL (cannot beat current best solution) using a cost bound. B&B; is used for optimization problems; backtracking for decision/enumeration.

4.2 N-Queens Problem

Problem: Place N queens on an N×N chessboard such that no two queens attack each other. Two queens attack each other if they are in the same row, same column, or same diagonal.

Key insight: Since we place exactly one queen per row, we only need to track which COLUMN each queen is in. $col[i]$ = column of queen placed in row i.

Conflict check between queen in row i and queen in row j:

- Same column: $col[i] = col[j]$
- Same diagonal: $|col[i] - col[j]| = |i - j|$

Algorithm:

- Start with row 1. Try placing queen in each column (1 to N).
- For each column: check if placement conflicts with any previously placed queen.
- If no conflict: place queen, move to next row.
- If all columns tried and all conflict: backtrack to previous row, move that queen to next column.
- If queen placed in row N without conflict: solution found.

4-Queens Solutions (memorise both):

Solution 1: $col = [_, 2, 4, 1, 3]$ (queen in row 1 at col 2, row 2 at col 4, row 3 at col 1, row 4 at col 3)

Solution 2: $col = [_, 3, 1, 4, 2]$

Draw the state-space tree showing all attempted placements and pruned branches — examiners look for this.

4.3 Graph Coloring

Problem: Color all vertices of a graph using at most m colors such that no two adjacent vertices (connected by an edge) share the same color. The **chromatic number** of a graph is the minimum number of colors needed.

Algorithm:

- Assign color 1 to vertex 1.
- For each subsequent vertex: try assigning colors 1, 2, ..., m in order.
- A color c is valid for vertex v if none of v 's neighbours have color c .
- If a valid color is found, assign it and move to the next vertex.
- If no valid color exists (all m colors conflict): backtrack to the previous vertex and try the next color for it.

Real-world applications: Map coloring (adjacent countries different colors). Register allocation in compilers. Scheduling problems (time slots for exams so no student has two at once).

In exam: Show a vertex-by-vertex assignment table and the state-space tree showing backtracking steps.

4.4 Subset Sum Problem

Problem: Given a set S of positive integers and a target sum X , find all subsets of S that sum exactly to X .

Example from slides: $S = \{3, 4, 5, 6\}$, $X = 9$.

Solutions: $\{3, 6\}$ ($3+6=9$) and $\{4, 5\}$ ($4+5=9$)

Algorithm — Implicit Binary Tree:

- At each level of the tree, decide whether to INCLUDE (left branch) or EXCLUDE (right branch) the current element.
- Prune (do not expand) a node if: current sum $> X$ (exceeded target) OR current sum + remaining elements $< X$ (cannot reach target even with all remaining).
- When a leaf is reached with sum $= X$: record as a solution.

Draw the binary tree with clear labels: show the running sum at each node, mark pruned nodes with X , and circle solution nodes.

4.5 Hamiltonian Cycle

Hamiltonian Path: A path in a graph that visits every vertex exactly once. **Hamiltonian Cycle:** A Hamiltonian path that also returns to the starting vertex. Not every graph has a Hamiltonian cycle. This is an NP-complete problem.

vs Eulerian: Eulerian path visits every EDGE exactly once. Hamiltonian visits every VERTEX exactly once.

Algorithm (backtracking):

- Start at vertex 0. Try adding each unvisited vertex that is adjacent to the current last vertex.
- If at position n (all vertices visited) and last vertex connects back to start → Hamiltonian cycle found.
- If no valid next vertex exists, backtrack.

No efficient (polynomial) algorithm exists for the general case. Used in TSP (next topic).

4.6 Travelling Salesman Problem (TSP) — Branch & Bound

Problem: Given n cities and distances between every pair, find the shortest route that visits every city exactly once and returns to the starting city. NP-hard problem — Branch and Bound finds the optimal solution more efficiently than brute force.

Cost Matrix Reduction (key step):

- For each ROW: subtract the minimum value in that row from all elements in the row.
- For each COLUMN: subtract the minimum value in that column from all elements.
- Sum of all subtracted values = LOWER BOUND for the starting (root) node.

Algorithm:

- Root node cost = matrix reduction cost.
- To expand node by taking edge (i,j): set entire row i and column j to infinity. Set $\text{matrix}[j][0]$ to infinity (cannot return to start early). Reduce again. New node cost = parent cost + $\text{matrix}[i][j]$ + new reduction cost.
- Expand the node with the MINIMUM cost first (best-first search).
- Prune any node whose cost $\geq$ current best solution found.
- When a leaf node is reached (all cities visited and path returns to start), update best solution.

Show the cost matrix at each step in exam. State the lower bound clearly.

UNIT 5 — Advanced Data Structures

5.1 Red-Black Trees

A **Red-Black Tree** is a self-balancing Binary Search Tree where every node is coloured RED or BLACK. The colouring ensures the tree remains approximately balanced, guaranteeing $O(\log n)$ time for all operations in the WORST case — unlike plain BST which is $O(n)$ worst case.

The 5 Properties (MUST memorise all 5):

Property 1: Every node is either RED or BLACK.

Property 2: The ROOT node is always BLACK.

Property 3: All LEAF nodes (NIL / sentinel nodes) are BLACK.

Property 4: If a node is RED, then BOTH its children must be BLACK. (Equivalently: no two consecutive red nodes on any path from root to leaf.)

Property 5: For every node, all simple paths from that node to descendant leaf nodes contain the same number of BLACK nodes. (This number is called the BLACK-HEIGHT of the node.)

Black-height $bh(x)$: Number of black nodes on any path from node x (exclusive) down to a leaf. Due to Property 5, all paths give the same count.

Height theorem: A Red-Black Tree with n internal nodes has height $h \leq 2 \times \log_2(n+1)$. This guarantees $O(\log n)$ for all operations.

Sentinel $nil[T]$: A single shared BLACK node representing ALL NIL leaves. Simplifies algorithms by allowing uniform treatment of all "null" children.

Rotations: Like AVL, RB Trees use Left-Rotate and Right-Rotate to rebalance. **Left-Rotate(x):** x 's right child y becomes the new subtree root; x becomes y 's left child; y 's old left child becomes x 's right child. **Right-Rotate(y):** The mirror image — y 's left child x becomes root; y becomes x 's right child.

RB Tree vs AVL Tree: AVL is more strictly balanced (difference in heights ≤ 1) $\rightarrow$ faster lookups. RB Tree rebalances less aggressively $\rightarrow$ faster insertions and deletions. RB Trees preferred in practice (e.g., `std::map` in C++, Java `TreeMap`).

5.2 B+ Trees

A **B+ Tree** is a variant of the B-Tree optimized for database indexing and file systems. The key difference: in a B+ Tree, ALL data records are stored ONLY in the leaf nodes. Internal nodes store only keys for routing/navigation.

B-Tree vs B+ Tree — Key Differences:

B-Tree

- Data stored in BOTH internal and leaf nodes
- Search may stop at internal node if key found
- No linking between leaf nodes
- Harder to do range queries
- Each key appears once in tree

B+ Tree

- Data stored ONLY in leaf nodes
- Internal nodes hold only keys for routing
- Leaf nodes linked as doubly-linked list
- Efficient range queries — traverse leaf list
- Keys duplicated: appear in internal nodes AND leaves

Why B+ Trees dominate in databases: Full table scan = just traverse the leaf linked list. Range queries (e.g., $10 \leq \text{age} \leq 30$) are very efficient. All leaf nodes at same level → predictable access time. Used by: MySQL InnoDB, PostgreSQL, SQLite, Oracle, MongoDB.

5.3 Tries (Prefix Trees)

A **Trie** (pronounced "try") is a tree-like data structure for storing strings where each path from root to a node represents a prefix of a stored string. Unlike BST where each node stores a complete key, each NODE in a trie stores a single CHARACTER.

Structure: Each node has up to 26 children (for lowercase English), one per character. A boolean "end-of-word" flag marks nodes where valid words end. The root represents the empty string (no character).

Insert(word): For each character in word, follow the edge if it exists; else create a new node. After inserting all characters, set end-of-word flag on the last node.

Search(word): Follow edges for each character. If any edge is missing → word not in trie. If all edges followed and end-of-word flag is true → found. If flag is false → word is just a prefix of another stored word, not present itself.

Time Complexity: $O(L)$ for both insert and search, where L = length of the string. Independent of number of strings stored.

Space Complexity: $O(\text{ALPHABET_SIZE} \times L \times N)$ where N = number of strings.

Applications: Autocomplete (search engines, keyboards). Spell checking. IP routing (Patricia Trie). Phone directories. Word games (Scrabble, Boggle solvers).

5.4 Binomial and Fibonacci Heaps

Both are advanced heap structures that support all standard heap operations but with better amortized complexities than binary heaps, especially for the merge (union) operation.

Binomial Tree B_k (recursive definition):

- B_0 = single node.
- B_k = two B_{k-1} trees linked together: one becomes the leftmost child of the other's root.
- Properties: B_k has 2^k nodes. Height = k . Root has degree k . Root's children are B_{k-1} , B_{k-2} , ..., B_1 , B_0 .

Binomial Heap: A collection of binomial trees satisfying the min-heap property, where no two trees have the same order. Analogous to binary representation: n nodes $\rightarrow$ trees present correspond to the 1-bits in $\text{binary}(n)$.

Fibonacci Heap: Even more relaxed structure. Trees need not be binomial. Lazy operations — only consolidate when extract-min is called. Key advantage: Decrease-key is $O(1)$ amortized (vs $O(\log n)$ in binary/binomial heap). This makes Dijkstra's algorithm run in $O(E + V \log V)$ with Fibonacci heaps.

Heap Operation Complexity Comparison:

Operation	Binary Heap	Binomial Heap	Fibonacci Heap
Insert	$O(\log n)$	$O(\log n)$	$O(1)$
Extract-Min	$O(\log n)$	$O(\log n)$	$O(\log n)$
Decrease-Key	$O(\log n)$	$O(\log n)$	$O(1)$ amort.
Union/Merge	$O(n)$	$O(\log n)$	$O(1)$
Find-Min	$O(1)$	$O(\log n)$	$O(1)$

Quick Reference — Formulae & Complexities

TREES

Max nodes at level l	2^l (root is level 0)
Max nodes in BT, height h	$2^{(h+1)} - 1$
Min height for N nodes	$\text{floor}(\log_2(N+1)) - 1$
Full BT theorem	$L = I+1 \mid N = 2I+1 \mid N = 2L-1$
Heap Sort time	$O(n \log n)$ – worst, best, average ALL the same
B-Tree order m : keys per non-root	Min: $\text{ceil}(m/2)-1$ Max: $m-1$
Spanning tree edges	Always $N-1$ edges for N vertices

GRAPHS

Kruskal's time	$O(E \log E)$
Prim's time	$O(V^2)$ with matrix $O(E \log V)$ with min-heap
Dijkstra time	$O(V^2)$ – no negative edges allowed
Bellman-Ford time	$O(V \times E)$ – handles negative edges, detects negative cycles
Floyd-Warshall time	$O(V^3)$ Update: $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$
DFS / BFS time	$O(V+E)$ for both

DYNAMIC PROGRAMMING

Knapsack recurrence	$c[i,w] = \max(c[i-1,w], v_i + c[i-1, w-w_i])$ if $w_i \leq w$
Knapsack time/space	$O(nW)$ time $O(nW)$ space
LCS recurrence (match)	$L[i,j] = L[i-1,j-1] + 1$ when $X[i] = Y[j]$
LCS recurrence (no match)	$L[i,j] = \max(L[i-1,j], L[i,j-1])$ when $X[i] \neq Y[j]$
LCS time	$O(m \times n)$
MCM recurrence	$m[i,j] = \min \text{ over } k: m[i,k] + m[k+1,j] + p[i-1] \times p[k] \times p[j]$
MCM time	$O(n^3)$
Floyd-Warshall (DP context)	Same as graph – $O(V^3)$, all-pairs shortest path

BACKTRACKING

N-Queens diagonal check	$ \text{col}[i] - \text{col}[j] = i - j $ implies same diagonal
4-Queens solutions	(2,4,1,3) and (3,1,4,2)

ADVANCED DS

RB Tree height bound	$h \leq 2 \times \log_2(n+1)$ where n = internal nodes
Black-height $bh(x)$	Black nodes from x to leaf (NOT counting x)
Binomial tree B_k nodes	2^k nodes, height k , root degree k
Trie search/insert time	$O(L)$ where L = length of the string
B-Tree order m : non-root	At least $\lceil m/2 \rceil - 1$ keys; at most $m-1$ keys
Cayley's formula	$N^{(N-2)}$ spanning trees in complete graph K_N

10-Mark Answer Structure — Use Every Time

For ANY 10-mark question, always follow this structure:

Step 1 — Definition: 2-3 lines. What is this topic? Give a precise definition.

Step 2 — Properties / Types: Bullet points listing key properties, or types if applicable.

Step 3 — Algorithm / Recurrence: Numbered steps for algorithm OR the recurrence formula for DP.

Step 4 — Worked Example with Diagram: DRAW IT. Show the tree / table / graph trace step by step. Label everything.

Step 5 — Time & Space Complexity: State both. Brief justification if possible.

Diagrams are the #1 thing that separates 8/10 from 5/10. Bring a ruler. Draw neat tables for DP problems.

All 5 units covered. Practise the numericals — that is what the marks are made of. Good luck.